

Rapport — Étape 1 : correction des failles de sécurité

Application Fun Kids Animation · Corrections, vérifications et procédure de test · 24 septembre 2026

Élément	Détail
Étape du plan d'action	Étape 1 — Corriger les failles critiques (audit du 24/09)
Branche / commit	nassim · commit du code : e8ed796
Fichiers modifiés	users/urls.py, availability/views.py, payments/views.py, web/views.py, reviews/views.py + tests associés
Base de données	Aucune modification — ni migration, ni changement de structure
Tests automatiques	79 tests : tous OK (56 existants + 23 nouveaux)
Résultat global	Étape 1 terminée

1. Objectif

Supprimer les failles de sécurité relevées par l'audit du 24/09 avant toute démonstration, sans dégrader les fonctionnalités existantes. Objectifs précis :

- Empêcher toute connexion à un compte sans mot de passe ni identité Google vérifiée.
- Protéger les indisponibilités des animateurs contre toute modification anonyme.
- N'accepter que les notifications de paiement réellement émises et signées par Stripe.
- Cloisonner les données : un client ne voit et ne modifie que ses propres réservations et avis.
- Prouver chaque correction par un test automatique et par un essai sur le serveur.

2. Commits associés

Commit	Date	Message	Contenu
e8ed796	24/09/2026 21:17	fix: correction des failles de securite critiques (etape 1)	Corrections du code (5 fichiers) + 23 tests de non-régression (4 fichiers)
8fcf7fa	24/09/2026 21:20	docs: ajout du rapport PDF de l'etape 1 (correction securite)	Première version de ce rapport
33dd562	24/09/2026 21:26	docs: mise a jour du rapport de l'etape 1	Ajout des commits, résultats obtenus, objectifs détaillés
(suivant)	24/09/2026	docs: parcours de test detaille dans les rapports	Version actuelle : parcours de test pas à pas avec adresses (port 8010)

Branche : nassim · Commit de référence pour le code corrigé : e8ed796. Consulter le détail : `git show e8ed796 --stat`

3. Récapitulatif des corrections

#	Faille corrigée	Gravité	Statut
1	Connexion à n'importe quel compte via l'URL de retour Google	Critique	✓ Réalisé
2	API des indisponibilités modifiable sans connexion	Critique	✓ Réalisé
3	Webhook Stripe accepté sans signature	Haute	✓ Réalisé

#	Faible corrigée	Gravité	Statut
4	Pages de retour de paiement sans contrôle du titulaire	Moyenne	✓ Réalisé
5	Avis modifiables par tout utilisateur connecté	Moyenne	✓ Réalisé

4. Description détaillée

4.1 Connexion Google (critique)

Avant

/auth/google-callback/?email=... ouvrait directement la session du compte indiqué, y compris administrateur, sans mot de passe ni vérification.

Après

L'adresse e-mail provient désormais **uniquement** de l'identité vérifiée par Google (échange du code d'autorisation, contrôle de email_verified). Les paramètres de l'URL sont ignorés. Le sélecteur de compte simulé (démonstration) n'est plus actif par défaut : il nécessite le mode DEBUG et l'activation explicite GOOGLE_LOGIN_SIMULATION=True dans le .env, n'accepte que l'envoi du formulaire, et refuse toujours les comptes administrateur et animateur. Sans configuration Google, le bouton renvoie vers la page de connexion avec un message clair.

Fichier : users/urls.py

4.2 API des indisponibilités (critique)

Avant

/api/availability/ était en accès libre : un visiteur anonyme pouvait créer, modifier ou supprimer les indisponibilités des animateurs, et donc fausser les créneaux proposés.

Après

Lecture toujours publique (liste, vérification de créneau, créneaux du jour, animateurs disponibles), nécessaire au tunnel de réservation. Création, modification et suppression réservées à l'administration. Les animateurs continuent de gérer leurs indisponibilités depuis leur espace personnel (inchangé).

Fichier : availability/views.py

4.3 Webhook Stripe (haute)

Avant

Sans STRIPE_WEBHOOK_SECRET, le webhook traitait tout événement reçu : un faux message « paiement réussi » suffisait à confirmer une réservation.

Après

Tout événement est désormais vérifié par sa signature Stripe. Sans secret valide, ou avec une signature fautive, la requête est refusée (HTTP 400) et journalisée. La confirmation du paiement par la page de succès, qui interroge directement Stripe, fonctionne toujours sans webhook.

Fichier : payments/views.py

4.4 Pages de retour de paiement (moyenne)

Avant

Les pages de succès et d'annulation affichaient le détail d'une réservation (adresse, enfant, montant) à partir de son seul numéro, sans vérifier l'utilisateur.

Après

Le détail n'est affiché qu'au titulaire de la réservation ou à l'administration. La mise à jour du statut après un paiement vérifié par Stripe reste effectuée dans tous les cas.

Fichier : web/views.py

4.5 Avis clients (moyenne)

Avant

Tout utilisateur connecté pouvait modifier ou supprimer n'importe quel avis, et en créer sur la réservation d'un autre client.

Après

Création et modification limitées au titulaire de la réservation (ou à l'administration). La lecture des avis reste publique.

Fichier : reviews/views.py

5. Vérifications effectuées

5.1 Tests automatiques

Suite	Nouveaux tests	Résultat
users/tests.py	9 — connexion Google : URL, formulaire, simulation, comptes privilégiés, identité vérifiée	✓ Réalisé
availability/tests.py	4 — anonyme, client, administrateur, lecture publique	✓ Réalisé
payments/tests.py	6 — webhook sans secret / signature fausse, pages de retour (titulaire, autre, anonyme)	✓ Réalisé
reviews/tests.py	4 — autre utilisateur, réservation d'autrui, titulaire, lecture publique	✓ Réalisé
Suite complète	79 tests (dont les 56 existants, adaptés pour le webhook signé)	✓ Réalisé

Contrôles complémentaires : manage.py check sans erreur, makemigrations --check : aucune migration.

5.2 Rejeu des attaques sur le serveur réel

Serveur de vérification lancé en local sur **http://127.0.0.1:8011** (base de données du projet), puis arrêté après les essais.

Essai	Avant	Après	Statut
GET /auth/google-callback/?email=animateur@...	302, session ouverte	302 vers /auth/login/, aucune session	✓ Réalisé
GET /auth/google-login/ (sans configuration)	faux sélecteur Google	302 vers /auth/login/ + message	✓ Réalisé
POST anonyme /api/availability/	201 (création)	403	✓ Réalisé
DELETE anonyme /api/availability/1/	autorisé	403	✓ Réalisé
GET /api/availability/slots/	200	200 (inchangé)	✓ Réalisé
POST webhook non signé « paiement réussi »	200, réservation confirmée	400, rien de modifié	✓ Réalisé
Page /payment-cancelled/?booking_id=1 anonyme	détail affiché	détail masqué	✓ Réalisé

5.3 Non-régression

Parcours	Résultat	Statut
Pages publiques : /, /nos-surprises/, /pricing/, /gallery/, /about/, /contact/	HTTP 200	✓ Réalisé
Connexion / inscription : /auth/login/, /auth/signup/, /admin/login/	HTTP 200	✓ Réalisé

Parcours	Résultat	Statut
Connexion par e-mail + mot de passe (animateur de démonstration)	302 vers l'accueil, tableau de bord 200	✓ Réalisé
API publiques : /api/services/, /api/reviews/	HTTP 200	✓ Réalisé

6. Résultats obtenus

Sortie de la suite de tests complète, relancée sur le commit final :

```
$ venv/bin/python manage.py test
Found 79 test(s).
System check identified no issues (0 silenced).
Ran 79 tests in 86.417s
```

OK

Sortie des contrôles de configuration :

```
$ venv/bin/python manage.py check
System check identified no issues (0 silenced).
$ venv/bin/python manage.py makemigrations --check --dry-run
No changes detected
```

Objectif	Résultat obtenu	Statut
Aucune connexion sans identité vérifiée	URL Google piégée → retour à la page de connexion, aucune session ouverte	✓ Réalisé
Indisponibilités protégées	Création / suppression anonymes → HTTP 403 ; lecture des créneaux → HTTP 200	✓ Réalisé
Paiements authentiques uniquement	Faux « paiement réussi » → HTTP 400, réservation inchangée	✓ Réalisé
Données cloisonnées	Réservation d'autrui masquée sur les pages de paiement ; avis d'autrui → HTTP 403	✓ Réalisé
Aucune régression	79/79 tests OK, pages publiques et connexion par mot de passe fonctionnelles	✓ Réalisé
Base de données intacte	Aucune migration, aucun changement de structure	✓ Réalisé

Conclusion : l'étape 1 est terminée et stable. Les failles relevées par l'audit ne sont plus exploitables et l'application fonctionne comme avant pour les parcours légitimes.

7. Comment tester soi-même

7.1 Tests automatiques (depuis la racine du projet)

```
venv/bin/python manage.py test
```

Résultat attendu : « Ran 79 tests ... OK ». Pour ne lancer que les tests de sécurité :

```
venv/bin/python manage.py test users availability payments reviews
```

7.2 Tests manuels pas à pas (port 8010)

Adresses d'accès (serveur lancé sur le port 8010)

Page	Adresse à ouvrir dans le navigateur
Accueil	<code>http://192.168.1.100:8010/</code>
Connexion (login)	<code>http://192.168.1.100:8010/auth/login/</code>
Inscription	<code>http://192.168.1.100:8010/auth/signup/</code>
Réserver	<code>http://192.168.1.100:8010/booking/</code>
Mon espace (client / animateur)	<code>http://192.168.1.100:8010/dashboard/</code>
Paieement d'une réservation	<code>http://192.168.1.100:8010/paiement/<numéro>/</code>
Administration	<code>http://192.168.1.100:8010/admin/</code>

192.168.1.100 est l'adresse du serveur sur le réseau local : à utiliser depuis un autre poste. Sur le serveur lui-même, remplacer par 127.0.0.1. Le serveur se lance depuis la racine du projet avec : `venv/bin/python manage.py runserver 0.0.0.0:8010`

Parcours de test

N°	Où aller	Action	Résultat attendu
1	Terminal du serveur	Lancer <code>venv/bin/python manage.py runserver 0.0.0.0:8010</code>	« Starting development server at <code>http://0.0.0.0:8010/</code> »
2	<code>/auth/google-callback/?email=admin@funkidz.fr</code>	Ouvrir cette adresse sans être connecté	Retour sur la page de connexion ; le menu affiche toujours « Connexion » (aucune session ouverte)
3	<code>/auth/login/</code>	Cliquer sur « Continuer avec Google »	Message « La connexion avec Google n'est pas disponible pour le moment. »
4	<code>/auth/login/</code>	Se connecter avec <code>client2@funkidz.fr</code> et son mot de passe	Redirection vers l'accueil, connecté (connexion normale inchangée)
5	Terminal	<code>curl -X DELETE http://127.0.0.1:8010/api/availability/1/</code>	Réponse 403 « Informations d'authentification non fournies »
6	<code>/payment-cancelled/?booking_id=1</code>	Ouvrir en fenêtre privée (non connecté)	Aucun détail de réservation affiché
7	Terminal	<code>curl -X POST -H 'Content-Type: application/json' -d '{}'</code> <code>http://127.0.0.1:8010/api/payments/webhook/</code>	Réponse 400 : webhook non signé refusé

8. Configuration (.env) et points d'attention

Variable	Rôle	Valeur par défaut
<code>GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET</code>	Connexion Google réelle (les deux sont requis)	vide → bouton Google indisponible
<code>GOOGLE_LOGIN_SIMULATION</code>	Sélecteur Google simulé pour une démo locale (DEBUG uniquement, comptes clients)	désactivé
<code>STRIPE_WEBHOOK_SECRET</code>	Obligatoire pour que le webhook Stripe soit accepté (production)	vide → webhook refusé

- Changement de comportement volontaire : le faux sélecteur « Choisir un compte Google » n'apparaît plus par défaut. Pour le réafficher en démonstration locale : `GOOGLE_LOGIN_SIMULATION=True` dans le `.env`.
- Ces variables ne sont pas encore décrites dans `.env.example` : ce fichier contient une modification en cours non committée (`ALLOWED_HOSTS`), laissée intacte. À compléter lors du prochain commit de configuration.
- Aucune clé ni secret n'a été ajouté au dépôt.

9. Suite du plan d'action

Étape	Contenu	Statut
1	Faillles de sécurité critiques	✓ Réalisé
2	Comptes et connexion multi-clients (e-mails en minuscules, comptes de test)	À faire
3	Parcours de paiement Stripe (Link, échec, annulation)	À faire
4	Brevo et e-mails de bout en bout	À faire
5 à 10	Créneaux, administration, calendrier, livraison du 29/09, soutenance	À faire